zensar

ZENSAR CONTENT STUDIO

# Pipeline Explanation

This document explains, step by step, what happens inside the app when you press a button, and which technology does each step. It is written for readers who are not programmers.

# Contents

0. Starting the app (start.bat)

1. Every pipeline starts the same way: reading

2. Projects pipeline (documents → slides, visuals, video)

3. Research Papers pipeline (paper → publisher format)

4. Brand Documents pipeline (any document → branded white paper)

5. Where the results go

This document explains, step by step, what happens inside the app when you press a button, and which technology does each step. It is written for readers who are not programmers.

## 0. Starting the app ( `start.bat` )

> `start.bat` → `bootstrap.ps1` → [checks + installs] → starts the server → opens the browser

1. **Already running?** If Content Studio already answers on the port, it just opens the browser. If another program uses the port, it picks the next free one.
2. **Visual C++ runtime:** installed if missing (PyTorch and ONNX Runtime need it).
3. **Python:** an installed Python 3.10–3.12 is found, or Python 3.12 is installed (winget, or the python.org installer on Windows Server). A private environment `.venv` is created.
4. **Network:** a proxy set in Windows or Git is detected and passed on. If pypi.org cannot be reached, the **offline bundle** is cloned from GitHub. It is reassembled from pieces under 100 MB, each with a SHA-256 checksum check.
5. **Packages:** `requirements.txt` is installed, either online or from the bundle.
6. **Models:** the text-understanding model (ONNX) is installed.
7. **Interface:** built with Node.js, or the ready-built version in `frontend/dist` is used.
8. **Image AI:** `nvidia-smi` detects a GPU. The CUDA or CPU build of PyTorch is installed, then `requirements-images.txt` and the DreamShaper model.
9. **Writing AI:** Ollama plus `qwen2.5:7b` (8 GB+ GPU) or `qwen2.5:3b`. This step is optional.
10. **System check**, then the server starts and the browser opens.

## 1. Every pipeline starts the same way: reading

> `file` → `reader` → clean structured content (sections, paragraphs, lists, tables, figures, formulas)

| File type | Technology | What the reader does |
|---|---|---|
| PDF | **PyMuPDF** | Reads each text piece with its position, size and font. Then it rebuilds the reading order across columns, removes page headers and footers, joins words split by hyphens, finds headings by size and weight, turns ruled tables into rows, and cuts figures and formulas out as pictures. |
| Word | **python-docx** | Uses Word's own heading styles, lists, tables and equations. |
| PowerPoint | **python-pptx** | Each slide becomes a page, and slide titles become headings. |
| Web page | **BeautifulSoup / lxml** | Reads the text (scripts are ignored). |
| Scanned PDF | **RapidOCR** (optional) | Reads text from the page image. |

| File type | Technology | What the reader does |
|---|---|---|
| Demo video | Scene detection + **qwen2.5vl** | Splits the video into scenes, describes each one and writes a narration. |
| Code repository | Own analyser | README, technology stack, components, API and architecture. |

## 2. Projects pipeline (documents → slides, visuals, video)

```
read → chunk → embed → rank by relevance → content contract → plan → verify → render → outputs
```

1. **Chunk** ( `intelligence/chunking.py` ): the text is cut into passages, and each remembers its page and section.
2. **Embed** ( `intelligence/embeddings.py` ): each passage becomes a list of numbers describing its meaning. This uses **all-MiniLM-L6-v2** on **ONNX Runtime**.
3. **Search and rank** ( `retrieval.py` , `relevance.py` ): **FAISS** finds the passages that match your instruction. Everything irrelevant is dropped.
4. **Content contract** ( `planning/content_contract.py` ): your instruction ("10 slides for executives") becomes a precise specification. Rules handle the clear parts; the local AI (**Qwen 2.5** via **Ollama**) interprets the rest.
5. **Plan** ( `planning/content_planner.py` ): slide by slide, the AI writes titles, bullets and narration from the selected passages only. Its answers must be valid **JSON**, checked with **Pydantic**.
6. **Verify** ( `intelligence/verification.py` ): every statement is checked against its source passage. Numbers must appear in the source, and unsupported claims are removed.
7. **Visuals** ( `planning/visual_planner.py` ): charts come only from real table numbers. Diagrams come from the document's own structure.
8. **Render** ( `rendering/` ):
   - one drawing description (scene) per slide, laid out on the brand grid with brand colours and fonts;
   - **python-pptx**: the editable PowerPoint;
   - **Pillow/Matplotlib**: PNG previews, charts and the PDF;
   - **SVG** vector images;
   - the video: motion graphics, narration (**Windows SAPI** voices), music and subtitles, encoded by **FFmpeg**.
9. **Outputs** are saved and shown in the project tabs, with a link from each statement to its source.

Without a local AI, step 5 uses **extractive mode**: real sentences from the document, unchanged.

## 3. Research Papers pipeline (paper → publisher format)

```
read → structure → references → citations → cross-references → format → write → report
```

1. **Read** the paper (section 1). PDF formulas whose fonts can't be read as text are kept as pictures.

2. **Structure** ( `papers/structure.py` ): title, authors, affiliations, e-mails, abstract, keywords, the section tree, and special sections (funding, acknowledgements…).
3. **References** ( `papers/references.py` ): the list is split into entries. Each entry is parsed by rules (IEEE, Springer, APA, Elsevier, ACM, Vancouver patterns) into authors, year, title, journal, volume, pages and DOI. A **.bib** file is used as it is. The local AI is only a fallback, and every field it returns must appear word for word in the original.
4. **Citations** ( `papers/citations.py` ): `[3]` , `[2–5]` and `(Smith et al., 2020)` are linked to their references.
5. **Cross-references** ( `papers/crossrefs.py` ): "Table 2", "Fig. 3", "Eq. (1)" and "Section 2.1" are linked, so they renumber in the new style.
6. **Format** ( `papers/cite_render.py` ): citations and the reference list are produced by **citeproc-py** with each publisher's official **CSL** style.
7. **Write** ( `papers/writers/` ):
   - a **LaTeX** project in the publisher's class (**Jinja2**);
   - **Word** (python-docx);
   - **PDF** (**ReportLab**);
   - **BibTeX**.
8. **Report** ( `papers/checks.py` ): the publisher's requirements (abstract length, keywords, statements) and the share of the original text preserved.

You can correct anything in the app and re-export to another publisher without reading the paper again.

# 4. Brand Documents pipeline (any document → branded white paper)

> `learn template (once) → read → fit length → plan images → generate images → lay out pages → check`

## 4.1 Learn the template (once per reference PDF) — `branddocs/reference.py`

Using **PyMuPDF**, the reference is studied three ways:

- **Text pieces** give the type sizes (headings, body, emphasis, labels, footer) and the page geometry (columns, margins).
- **Vector drawings** give the coloured cards and bands, corner radii, rules, heading dashes and bullets. The **logo, pattern and decorative shapes** are kept as exact paths.
- **The rendered page** (pixels, via **NumPy**) gives the true colours.

Each page is classified as **cover, intro (opening card), text page, image page, conclusion (closing card) or back cover**. The template also records:

- the footer pattern, with `{year}` and `{page}` filled in later;
- the company text on the back cover;
- the photo colour style;
- the content size: for example **1,200 words, 10 sections, 1 image page**.

### 4.2 Read the document — `branddocs/content.py`

The readers from section 1 are used in general-document mode:

- headings set over several lines are joined;
- drop-cap letters are fixed;
- drawn bullets become list items;
- labels like "Clarity:" are kept;
- the conclusion and an "Authored by" block are found.

### 4.3 Fit the length — `branddocs/fit.py`

- **Match the reference** (default), a **custom** word count, or the **full** text.
- The document is regrouped into as many parts as the reference has sections. Each part gets a share of the words.
- The local AI (**Qwen 2.5**) **condenses or expands** each part. Its rules: plain English; keep every key point, fact, figure and name; drop citations and author lists; add nothing.
- **Every number** in the result is checked against the source. A part that adds one is replaced by the source's own sentences.
- If the result is still too long, closing sentences are removed. Nothing is rewritten.
- Without the AI, the key sentences are selected instead (extractive).

### 4.4 Images — `branddocs/imagegen.py` + `imagegen_worker.py`

1. The local AI writes a short **scene description** for the cover and for each image section, from the content.
2. **Stable Diffusion** (DreamShaper 8 LCM, **Diffusers + PyTorch**) creates each picture in a separate process. This uses the GPU when one is available.
3. Each picture is **colour-matched** to the reference photos (CIELAB colour transfer with NumPy), enlarged and sharpened.
4. Pictures are **cached**, so "Create again" reuses them.

### 4.5 Lay out the pages — `branddocs/compose.py`

**ReportLab** draws:

- **Cover:** the image, darkened just enough behind the logo and title, the brand logo (exact vectors), the title and the label.
- **Intro:** the opening section in the coloured card, then the next section in two balanced columns.
- **Text pages:** two columns, headings with the brand dash, rules between sections, bullets, tables and formulas at their real size.
- **Image page:** a photo fading into the page, with one section per column.
- **Conclusion:** the closing card or band. Text is placed above it when the reference does that.
- **Back cover:** the authors, the brand pattern and logo, and the company text.
- **Footer** on every page, for example "© Zensar Technologies, 2026 | Page 3".
- If the text would spill a few lines onto an extra page, it is set slightly tighter instead.

### 4.6 Check — `branddocs/service.py`

The finished PDF is read back to confirm the words arrived. The report lists:

- how the length was fitted;
- any unsupported numbers;
- fonts used;
- images created.

## 5. Where the results go

| Tool | Folder |
|---|---|
| Projects | `outputs/<project>/` and `projects/<project>/` |
| Research Papers | `data/papers/<paper>/outputs/<format>/` |
| Brand Documents | `data/branddocs/docs/<document>/document.pdf` (templates in `data/branddocs/templates/`) |

Everything is also downloadable from the app.